

JAVA TEMELLERİ

Değişken Tanımlama ve Veri Türleri

Değişken Nedir?

- Programlama dilinde temel verileri saklamak ve bu verileri sonradan kullanmak için oluşturulan **bellek bölgeleridir.**
- Değişkenler hafızada geçici olarak saklanırlar.

Değişken Oluşturma

int
↓
Değişken Türü

deger
↓
Değişken Adı

= 100 ;
↓ ↓ ↓
Atama Aldığı Değer Kod Satırını
Operatörü Bitirme işareti

deger
100
deger ismi verilen değişkenin
bellekte tuttuğu alanda şu anda
100 değeri bulunmaktadır.

deger = 200; 200
deger = 50; 50

Java dilinde veri tipleri

- İLKEL (PRIMITIVE)
- REFERANS

tipleri olmak üzere 2'ye ayrılıyor.

Data Types In Java

Primitive Data Types

Boolean

1

Boolean

Character

2

char

Integer

3

byte

4

short

5

int

6

long

Float

7

float

8

double

Non-Primitive Data Types or Reference Data Types

Array

class

Interfaces

String

Enum

İlkel (Primitive) Veri Tipleri

- **TAM SAYILAR(Integers)**
 - byte (-128 ile +127 arası)
 - short (-32768 ile +32767 arası)
 - int (32bit) * En çok kullanılan veri tipleri
 - long(64bit) → long l= 10L;
- **ONDALIKLI SAYILAR(FLOATING NUMBERS)**
 - float(32bit) → float f= 10.5f;
 - double(64bit) → double d= 10.7d; /* d harfi sonuna eklenmeden de çalışabilir
- **KARAKTERLER(Characters)**
 - char → char c= 'c';
- **BOOLEAN(Characters)**
 - Boolean (True/False) : Doğru/Yanlış değerler tutarlar
 - → boolean b= true;

Referans Veri Tipleri

- **Kullanacağımız nesneleri ifade eder.**
 - Örneğin Object sınıfı
- **String bir referans tipidir. Ama Primitive(İlkel) değişkenler gibi çalışır**
 - `String s= "Mobil Programlama";`

Değişken Tanımlama Kuralları

- **Java büyük/küçük harf duyarlı bir dildir.(Case-Sensitive)**
 - Değişken isimlendirirken dikkat edilmelidir.
 - `int say;`
 - `int Say;`
birbirinden farklı değişkenlerdir.
- **Türkçe karakter bulunmaz**
 - **`String değer; // HATA`**
- **Rakam ile başlayamaz.**
 - `int 1sayi; // HATA`
- **Boşluk içeremez.**
 - `String ad soyad="Mobil Programlama"; // HATA`
- **Özel karakter içermez(_ hariç) ?????**
 - `String ad %="Mobil Programlama"; // HATA`
- **Sınıf isimleri büyük harfle başlar. (PascalCase notasyonu kullanılır)**
- **Değişken tanımlarken camelCase notasyonuna göre tanımlama yapılır.**

ÖRNEK

```
// tek satır açıklama için kullanılır
```

```
/*
```

```
Çok satırlı açıklamalar için kullanılır
```

```
*/
```

```
int sayi=50;
```

```
//int sayi=100; *HATA Aynı kapsama alanı içindeki tanımlanan bir seğişken bir daha tanımlanmamalıdır.
```

```
sayi=300;
```

```
//Değişkenler için camelCase notasyonu kullınılır
```

```
String aracMarka="A";
```

```
String aracModel="B";
```

```
int aracModelYil=2023;
```

```
double aracFiyat=1250000.50d;
```

```
boolean aracYenimi=true;
```

```
float aracMotorGucu=1.6f;
```

```
System.out.println("\u00A9 Araç Bilgileri \u00A9 ");
```

```
System.out.println("Araç Markası: \t"+aracMarka);
```

```
System.out.println("Araç Modeli: \t"+aracModel);
```

```
System.out.println("Araç Model Yılı: \t"+aracModelYil);
```

```
System.out.println("Araç Fiyatı: \t"+aracFiyat+"₺");
```

```
System.out.println("Araç Yenimi: \t"+aracYenimi);
```

```
System.out.println("Araç Motor Gücü: \t"+aracMotorGucu);
```

Tür Dönüşümü

- Sayısal değerlerin kendi aralarında dönüşümü
- Sayısal ifadelerin String ifadeye dönüşümü
- String ifadelerin Sayısal ifadeye dönüşümü

Sayısal ifadelerin dönüşümü

double→float→long→int→short→byte [Büyükten Küçüğe - Daralan]



byte→short→int→long→float→double [Küçükten Büyüğe - Genişleyen]



- **Küçük veri tipleri büyük veri tiplerine çevrilirken direk çevrilebilir**
- **Ondalıklı sayılar(float, double) tam sayılara çevrilirken ondalık kısmı atılır.**

Sayısal ifadelerin dönüşümü(Boxing - Unboxing)

- **//Küçük sayılardan büyük sayılara dönüşüm direk yapılabilir**
- **byte b=100;**
- **int i=b;**
- **double d=b;**
- **System.out.println("byte değeri: "+b);**
- **System.out.println("int değeri: "+i);**
- **System.out.println("double değeri: "+d);**

Sayısal ifadelerin dönüşümü(Boxing - Unboxing)

//Büyük sayılardan küçük sayılara dönüşüm aşağıdaki gibi yapılabilir

```
double d=100.50d;  
int i= (int)d;  
byte b=(byte)d;  
System.out.println("double değeri: "+d);  
System.out.println("int değeri: "+i);  
System.out.println("byte değeri: "+b);
```

ÖRNEK

//Küçük sayılardan büyük sayılara dönüşüm direk yapılabilir

```
byte b=100;  
int i=b;  
double d=b;  
//System.out.println("byte değeri: "+b);  
//System.out.println("int değeri: "+i);  
//System.out.println("double değeri: "+d);
```

//Büyük sayılardan Küçük sayılara dönüşüm

```
d = 120.5;  
i = (int)d;  
b = (byte)d;  
System.out.println("double değeri: "+ d);  
System.out.println("int değeri: "+ i);  
System.out.println("byte değeri: "+ b);
```

double sonuc = i/100; //iki tam sayı bölmesi yapılırsa değişken ondalıklı olsa bile sonuç tamsayı olur. Bölme işlemindeki elemanlardan en az biri ondalıklı olmalıdır

```
sonuc=i/100d;  
sonuc=(double)i/100;  
System.out.println(sonuc);
```

Sayısal ifadelerin String ifadelere dönüşümü

```
int deger = 100;
```

```
String s = String.valueOf(deger) //En çok kullanılan yöntem
```

```
s = Integer.toString(deger);
```

```
s = "Sayı: " + deger // Ekrana birleştirme yöntemiyle yazacaksak bu yöntem kullanılır
```

String ifadelerin Sayısal ifadelere dönüşümü

```
String s = "100";
```

```
int i = Integer.parseInt(s); //En çok kullanılan yöntem parse metodur  
i = Integer.valueOf(s);
```

```
double d = Double.parseDouble(s);
```

Örnek:

```
String say1="100";
```

```
String say2="55";
```

```
int s1,s2;
```

```
double sonuc;
```

```
s1=Integer.parseInt(say1);
```

```
s2=Integer.parseInt(say2);
```

```
sonuc=s1/(double)s2;
```

```
String sonucDegeri=String.valueOf(sonuc);
```

```
System.out.println(sonucDegeri);
```


Örnek

```
public class Main
{
    public static void main(String[] args) {
        //Sayısal ifadelerin Metinsel(String) ifadelere dönüşümü
        int adet=100;
        double fiyat = 10.5;
        double toplamFiyat= adet*fiyat;
        String adetBilgisi= String.valueOf(adet); //String.valueOf() metodu en sık kullanılan metoddur sayısal ifadeleri String ifadelere çevirirken
        String fiyatBilgisi = Double.toString(fiyat);
        String toplamDegerTutari="Toplam Değeri: " + toplamFiyat; //String birleştirmede sayısal değişkenin değeri otomatik Stringe çevrilir.
        System.out.println("Adet: " + adetBilgisi);
        System.out.println("Fiyat: " + fiyatBilgisi);
        System.out.println("Toplam Değer: " + toplamDegerTutari);

        //Metinsel(String) ifadelerin sayısala dönüşümü
        adetBilgisi="1000";
        fiyatBilgisi="20.25";
        adet = Integer.parseInt(adetBilgisi); // parse metodları en sık kullanılan stringten sayısala dönüşüm metodudur
        fiyat = Double.valueOf(fiyatBilgisi);
        toplamFiyat = adet * fiyat;

        adetBilgisi= String.valueOf(adet);
        fiyatBilgisi = String.valueOf(fiyat);
        toplamDegerTutari="Toplam Değeri: " + toplamFiyat;

        System.out.println("Adet: " + adetBilgisi);
        System.out.println("Fiyat: " + fiyatBilgisi);
        System.out.println("Toplam Değer: " + toplamDegerTutari);
    }
}
```

Java Özel Karakterler(Kaçış Karakterleri)

- Java'da Escape (Kaçış) karakterleri ile bazı özel durumlar durumlar gerçekleştirilir.
- Kaçış karakterleri (\) ters eğik çizgi ile ifade edilip sonrasında yazılan karakter ile özel işleve sahip olurlar.

- \t – tab(Sekme)
- \b – backspace (a step backward in the text or deletion of a single character)
- \n – new line(Yeni satır)
- \r – carriage return(Metne satır başı ekler)
- \f – form feed(Sayfa sonunu belirtmek için kullanılır)
- \' – single quote(Tek tırnak ekler)
- \" – double quote(Çift tırnak ekler)
- \\ – backslash

Kaçış Karakterleri	Açıklama
\t	tab boşluk ekler
\b	backspace ekler
\n	bir satır aşağı atlar
\r	metne bir satır başı ekler
\f	Eski bir tekniktir ve sayfa sonunu belirtmek için kullanılır.
\'	tek tırnak (') eklemek için kullanılır
\"	çift tırnak (") eklemek için kullanılır
\\	ters eğik çizgi eklemek için kullanılır

Java Özel Karakterler(Kaçış Karakterleri)-Kullanımı

- Java'da Escape (Kaçış) karakterleri ile bazı özel durumlar durumlar gerçekleştirilir.
- Kaçış karakterleri (\) ters eğik çizgi ile ifade edilip sonrasında yazılan karakter ile özel işleve sahip olurlar.
- \t – tab(Sekme)
- \b – backspace (a step backward in the text or deletion of a single character)
- \n – new line(Yeni satır)
- \r – carriage return(Metne satır başı ekler)
- \f – form feed(Sayfa sonunu belirtmek için kullanılır)
- \' – single quote(Tek tırnak ekler)
- \" – double quote(Çift tırnak ekler)
- \\ – backslash

Java Özel Karakterler(Unicode Özel Karakterler)

- Java'da Unicode özel karakterleri eklemek için **\u+<hexadecimal digit>** şeklinde bir kullanım yapılabilir.

ÖRNEK

\u00A9 : © karakterini gösterir

\u03A9 : Ω karakterini gösterir

Java Operatörler

- Aritmetik Operatörler
- Atama Operatörleri
- Karşılaştırma Operatörler
- Mantıksal Operatörler
- Bitisel Operatörler

Java Operatörler

Aritmetik Operatörler

Operator	Ad	Tanım	Örnek
+	Toplama	Değerleri toplarken kullanılır	$x + y$
-	Çıkarma	Çıkarma işleminde kullanılır	$x - y$
*	Çarpma	Değerleri çarparken kullanılır	$x * y$
/	Bölme	Bölme işleminde kullanılır	x / y
%	Mod Alma	Bölme işleminde kalanı verir	$x \% y$
++	Bir artırma	Değeri bir artırır($x=x+1$)	$++x$ veya $x++$
--	Bir Azaltma	*-/Değeri bir azaltır($x=x-1$)	$--x$ veya $x--$

Java Operatörler

Aritmetik Operatörler

```
1 int x=30;  
2 int y=3;  
3 System.out.println("Toplama işlemi = " + (x+y));  
4 System.out.println("Çıkarma işlemi = " + (x-y));  
5 System.out.println("Çarpma işlemi = " + (x*y));  
6 System.out.println("Bölme işlemi = " + (x/y));  
7 System.out.println("Mod Alma işlemi = " + (x%y));
```

```
1 Toplama işlemi = 33  
2 Çıkarma işlemi = 27  
3 Çarpma işlemi = 90  
4 Bölme işlemi = 10  
5 Mod Alma işlemi = 0
```

Java Operatörler

Atama Operatörleri

Operator	Örnek Kullanım	Aynı Kullanımın Farklı Yazımı
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
&=	x &= 3	x = x & 3
=	x = 3	x = x 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3

Java Operatörler

Atama Operatörleri

```
int a=21,b=13,c=160,d=55;  
a+=9; //a=a+9 yapısının kısa kullanımıdır  
b-=2; //b=b-2  
c*=3;  
d/=5;  
System.out.println(a);  
System.out.println(b);  
System.out.println(c);  
System.out.println(d);
```

Ekran Çıktısı

```
1 30  
2 11  
3 480  
4 11
```

Java Operatörler

İlişkisel(Karşılaştırma) Operatörleri

Operator	Ad	Örnek
==	Eşittir	x == y
!=	Eşit değil	x != y
>	Büyüktür	x > y
<	Küçüktür	x < y
>=	Büyük eşit	x >= y
<=	Küçük eşit	x <= y

Java Operatörler

İlişkisel(Karşılaştırma) Operatörleri - Örnek

```
int a = 60;
```

```
int b = 34;
```

```
System.out.println("Sayılar birbirine eşit midir? = " + (a == b) );
```

```
System.out.println("Sayılar birbirinden farklıdır? = " + (a != b) );
```

```
System.out.println("a sayısı b sayısından büyükmüdür? = " + (a > b) );
```

```
System.out.println("b sayısı a sayısından büyükmüdür? = " + (a < b) );
```

```
System.out.println("b sayısı a sayısından büyük veya eşitmidir? = " + (b >= a) );
```

```
System.out.println("b sayısı a sayısından küçük veya eşitmidir? = " + (b <= a) );
```

- 1 Sayılar birbirine eşit midir? = **false**
- 2 Sayılar birbirinden farklıdır? = **true**
- 3 a sayısı b sayısından büyükmüdür? = **true**
- 4 b sayısı a sayısından büyükmüdür? = **false**
- 5 b sayısı a sayısından büyük veya eşitmidir? = **false**
- 6 b sayısı a sayısından küçük veya eşitmidir? = **true**

Java Şart Deyimleri

If Yapısı

Kullanım Şekli-1

```
if (koşul) {  
    // Şart doğru ise çalıştırılacak kod bloğu  
}
```

Kullanım Şekli-2

```
if (koşul)  
    // Şart doğru ise çalıştırılacak kod satırı  
* Bu kullanım if yapısı altında sadece tek satır varsa kullanılabilir. Kırklangıç  
parantezler kullanmaya gerek yoktur
```

Örnek:

```
int x = 100;  
int y = 50;  
if (x > y) { //true  
    System.out.println("x sayısı y sayısından büyüktür");  
}
```

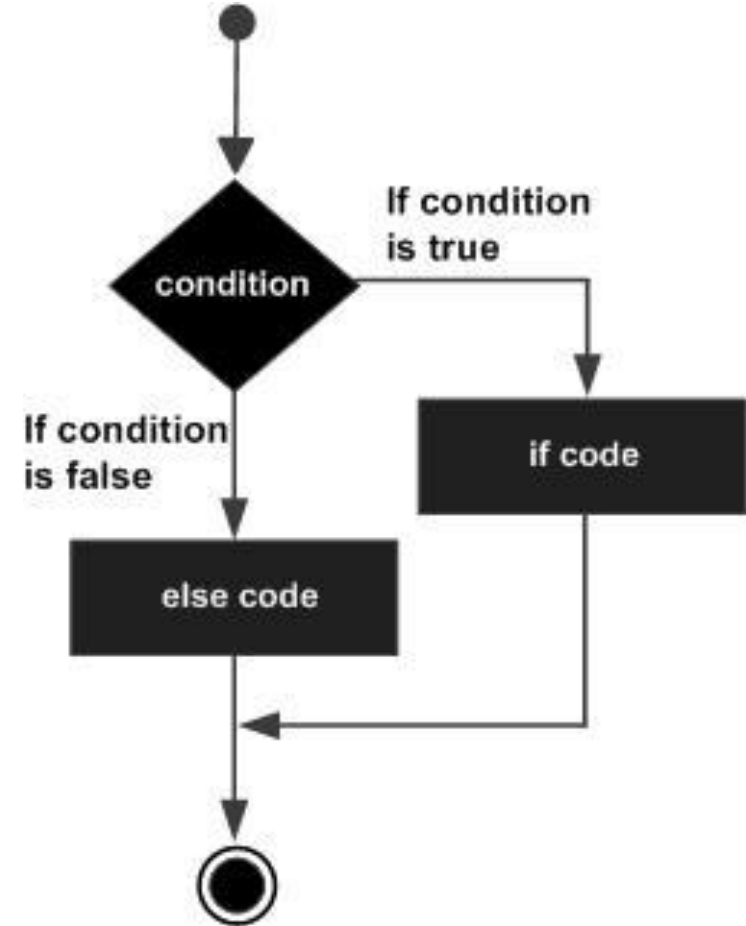
Çıktı: x sayısı y sayısından büyüktür

Java Şart Deyimleri

Else Yapısı

Koşul yanlış olduğu durumda çalışacak kodlar bu yapı altına yazılır.

```
if (koşul) {  
    // Şart doğru(true) ise çalıştırılacak kod bloğu  
}  
  
else {  
    // Şart yanlış(false) ise çalıştırılacak kod bloğu  
}  
  
int zaman = 20;  
if (zaman < 18) { //true  
    System.out.println("İyi günler.");  
}  
  
else { //false  
    System.out.println("İyi geceler."); }  
  
// Çıktı: «İyi geceler.» *şart yanlış olduğundan else bloğu çalışır
```



Java Şart Deyimleri

Else If Yapısı

İlk koşulun yanlış olması durumunda yeni bir koşulu(birden fazla olabilir) belirtmek için else if ifadesi kullanılır.

```
if (koşul1) {  
    // koşul1 doğru ise çalıştırılacak kod bloğu}  
  
else if (koşul2) {  
    // koşul1 yanlış ise çalıştırılacak kod bloğu ve koşul2 doğru ise çalıştırılacak kod bloğu  
}  
  
else {  
    // koşul1 yanlış ve koşul2 yanlış ise çalıştırılacak kod bloğu  
}
```

```
int time = 15;  
  
if (time < 10) {  
    System.out.println("Good morning.");  
}  
  
else if (time < 18) {  
    System.out.println("Good day.");  
}  
  
else {  
    System.out.println("Good evening.");  
}  
  
// Çıktı "Good day."
```

Java Şart Deyimleri

Kısa If Yapısı(ternary if)

Ayrıca, üç işlenenden oluştuğu için üçlü operatör olarak bilinen bir kısa if else vardır.

Birden fazla kod satırını tek bir satırla değiştirmek için kullanılabilir ve çoğunlukla basit if else ifadelerini değiştirmek için kullanılır.

değişken = (koşul) ? Koşul Doğru ise : koşul Yanlış ise;

```
int zaman = 20;

if (zaman < 18) {
    System.out.println("Good day.");
}
else {
    System.out.println("Good evening."); }
```

İfadesini tek satırda if kullanımı ile aşağıdaki gibi yazabiliriz.

```
int zaman = 20;

String sonuc = (zaman < 18) ? "Good day." : "Good evening.";

System.out.println(sonuc);

// Çıktı: Good evening." *şart yanlış olduğundan : sonraki değeri atar.
```

Java Şart Deyimleri

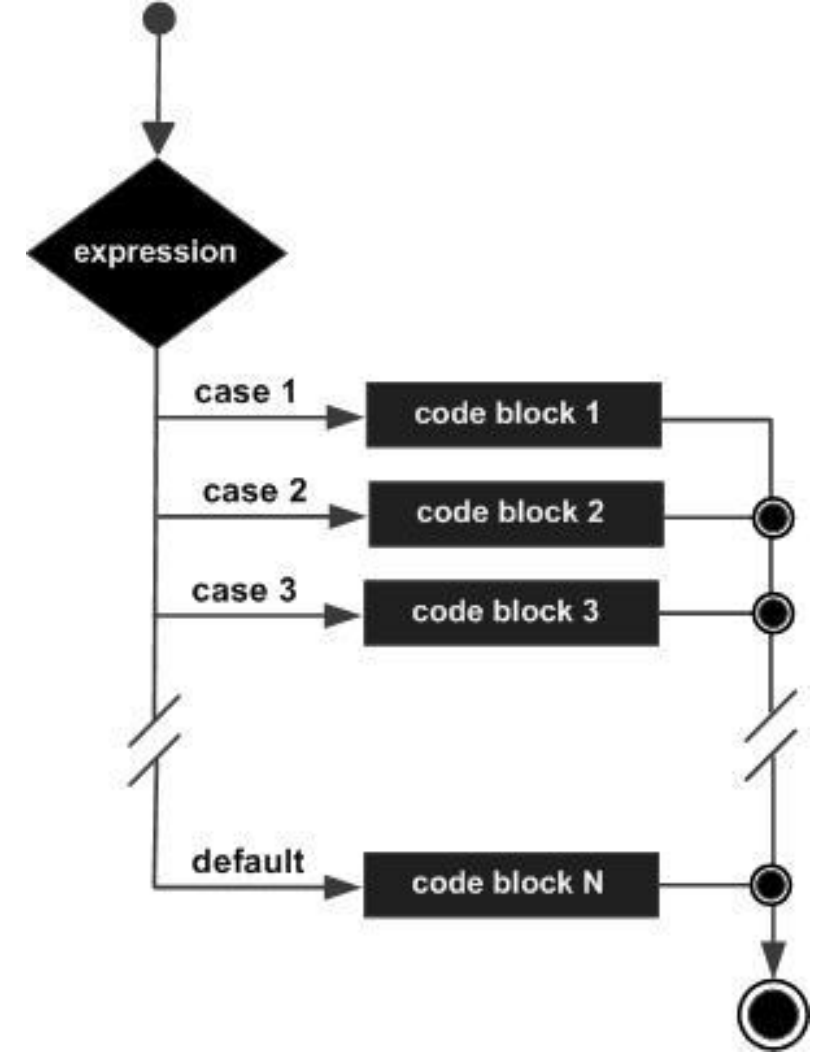
Switch Case Kullanımı

Çok sayıda if..else ifadesi yazmak yerine switch ifadesini kullanabilirsiniz.

```
switch(ifade) {  
  
    case x:  
        // kod bloğu  
        break;  
  
    case y:  
        // kod bloğu  
        break;  
  
    default:  
        // kod bloğu  
  
}
```

Nasıl çalışır:

- Switch ifadesi bir kez değerlendirilir.
- İfadenin değeri her bir durumun değerleriyle karşılaştırılır.
- Bir eşleşme varsa ilgili kod bloğu yürütülür.
- Break ve default anahtar sözcükleri isteğe bağlıdır.
- default herhangi bir eşleşme olmaması durumunda çalışması istenen kodların yazıldığı bir alandır.
- Break ifadesi kodu sonlandırır. Alt satırları işletmez.



Java Şart Deyimleri

Switch Case Kullanımı

Çok sayıda if..else ifadesi yazmak yerine switch ifadesini kullanabilirsiniz.

```
int day = 4;

switch (day) {

    case 1: //(4==1) //FALSE
        System.out.println("Monday");
        break;

    case 2:
        System.out.println("Tuesday");
        break;

    case 3:
        System.out.println("Wednesday");
        break;
    case 4:
        System.out.println("Thursday");
        break;

    case 5:
        System.out.println("Friday");
        break;

    case 6:
        System.out.println("Saturday");
        break;

    case 7:
        System.out.println("Sunday");
        break;

}
```

// Çıktı "Thursday" (day 4)

Java Şart Deyimleri

Mantıksal Operatörler

Operator	Adı	Description	Örnek Kullanım
&&	Mantıksal ve (Logical and)	Tüm ifadeler de doğruysa true değerini döndürür	(x < 5) && (y < 10)
	Mantıksal yada (Logical or)	İfadelerden biri doğruysa true değerini döndürür	(x < 5) (y < 4)
!	Mantıksal değil (Logical not)	Sonucu tersine çevirir, eğer sonuç doğruysa false değerini döndürür	!(x < 5 && y < 10)

```
a = 10, b = 20, c = 20
condition1: a < b //true
condition2: b == c //true
if(condition1 && condition2) //TRUE
d = a + b + c
// Since both the conditions are true
//d = 50.
```

```
a = 10, b = 20, c = 20
condition1: a < b //true
condition2: b > c //false
if(condition1 || condition2) //TRUE
d = a + b + c
// Since one of the condition is true
//d = 50.
```

```
a = 10, b = 20
!(a < b) // returns false
!(a > b) // returns true
```

Java Döngüler

For Yapısı

Döngü Nedir ?

Döngüler, belirli bir koşul sağlandığı sürece tekrarlanması gereken işler için kullanılan programlama kodlarıdır. Döngü deyimleri, bahsedilen komut parçalarının belirtilen şartlar gerçekleştikçe tekrar tekrar işlenmesini sağlar.

```
for(başlatma(1) ; koşul(2) ; arttırma/azaltma(3)){  
    //Çalıştırılacak Kodlar  
}
```

Döngü üç parametreden oluşur.

1. Başlatma: Kod bloğunun yürütülmesinden önce (bir kez) yürütülür.
2. Koşul: Kod bloğunun yürütülmesine ilişkin koşulu tanımlar.
3. Arttırma/Azaltma: Kod bloğu çalıştırıldıktan sonra (her zaman) yürütülür.

*** For döngüsünde tekrar sayısı sabittir.**

Java Döngüler

For Yapısı-Örnekler

Örnek Kullanım 1: Artan döngü +1

```
for (int i = 0; i < 5; i++) {  
    System.out.println(i);  
}
```

Ekran Çıktısı:

0
1
2
3
4

Örnek Kullanım 2: Artan Döngü +2

```
for (int i = 0; i <= 10; i = i + 2) {  
    System.out.println(i);  
}
```

Ekran Çıktısı:

0
2
4
6
8
10

Örnek Kullanım 3: Azalan Döngü -1

```
for (int i = 5; i > 0; i--) {  
    System.out.println(i);  
}
```

Ekran Çıktısı:

5
4
3
2
1

Java Döngüler

While Yapısı

- While döngülerinde, tekrar sayısı belli değildir.
- Döngü içerisinde oluşturmuş olduğumuz **şart boolean türde true olduğu sürece döngü devam eder**, bu true değer false değere döndüğü takdirde döngüden çıkılır ve program döngünün bittiği yerden çalışmaya devam eder.
- Anlamsal olarak “iken” anlamına gelen while kelimesini “x şartımız y değerinden küçük iken bu kodları çalıştır” şeklinde düşünebiliriz.

```
while (boolean Koşul){  
    //Çalıştırılacak Kodlar  
    Güncelleme bildirimi  
}
```

Java Döngüler

While Yapısı - Örnekler

Örnek Kullanım 1:

```
int i = 0;
while (i < 5) {
    System.out.println(i);

    i++;
}
```

Ekran Çıktısı:

```
0
1
2
3
4
```

Java Döngüler

Do While Yapısı

- While döngülerinden, tek farkı ilk adımda herhangi bir kontrol yapılmamasıdır. Kısacası bu blok içindeki işlem mutlaka bir kez çalışır
- Döngü içerisinde oluşturmuş olduğumuz **şart boolean türde true olduğu sürece döngü devam eder**, bu true değer false döndüğü takdirde döngüden çıkılır ve program döngünün bittiği yerden çalışmaya devam eder.
- Anlamsal olarak “iken” anlamına gelen while kelimesini “x şartımız y değerinden küçük iken bu kodları çalıştır” şeklinde düşünebiliriz.

```
do {  
    //Çalıştırılacak kodlar \ döngüsel gövde  
    //güncelleme bildirimi  
} while (boolean koşul);
```

Örnek Kullanım 1:

```
int i = 0;  
do{  
    System.out.println(i);  
    i++;  
}  
while (i < 5)
```

Ekran Çıktısı:

```
0  
1  
2  
3  
4
```

Java Döngüler

For each

- Bu döngü yapısı dizi veya koleksiyon içindeki elaman sayısını dikkate alarak çalışır. Döngü son elamana gelince otomatik sonlanır.

```
for (veri_türü degisken : Dizi veya Koleksiyon Adı){  
    //çalıştırılacak kodlar  
}
```

Örnek Kullanım 1:

```
String[] araclar = {"Volvo", "BMW", "Ford", "Mazda"};  
for (String arac : araclar) {  
    System.out.println(arac);  
}
```

Ekran Çıktısı:

```
Volvo  
BMW  
Ford  
Mazda
```

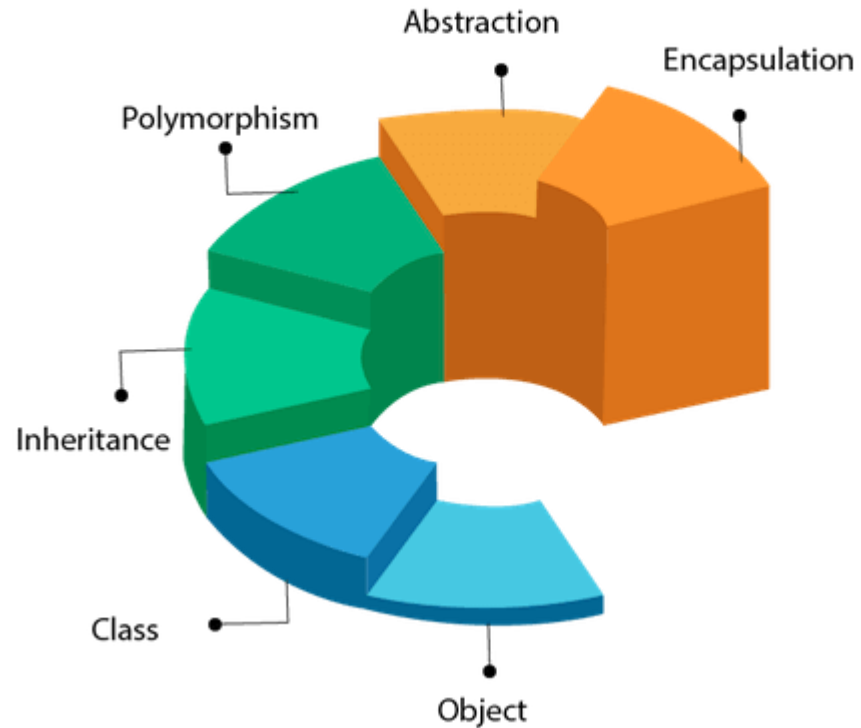

Java Metotlar

- İçerik

Java Sınıflar(Class)

- OOP(Object-Oriented Programming)(Nesne Tabanlı Programlama)

OOPs (Object-Oriented Programming System)



Java Sınıflar(Class)

(Object)Nesne

Java Nesnesi: Durumu ve davranışı olan her varlık nesne olarak bilinir.

Örneğin bir sandalye, kalem, masa, klavye, bisiklet vb. Fiziksel ya da mantıksal olabilir.

Bir Nesne bir sınıfın örneği olarak tanımlanabilir.

Bir nesne bir adres içerir ve bellekte biraz yer kaplar.

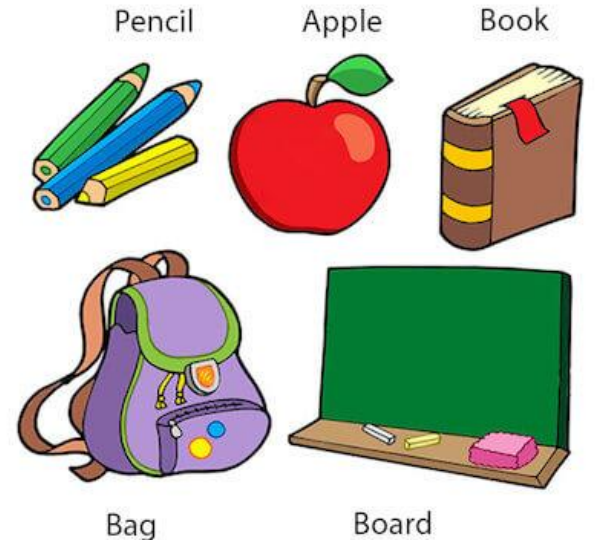
Nesneler birbirlerinin verilerinin veya kodlarının ayrıntılarını bilmeden iletişim kurabilir.

Gerekli olan tek şey, kabul edilen mesajın türü ve nesnelerin döndürdüğü yanıtın türüdür.

Örnek: Araba bir nesnedir çünkü rengi, modeli, markası vb. durumlarının yanı sıra hızlanması, görünümü vb. davranışları da vardır.



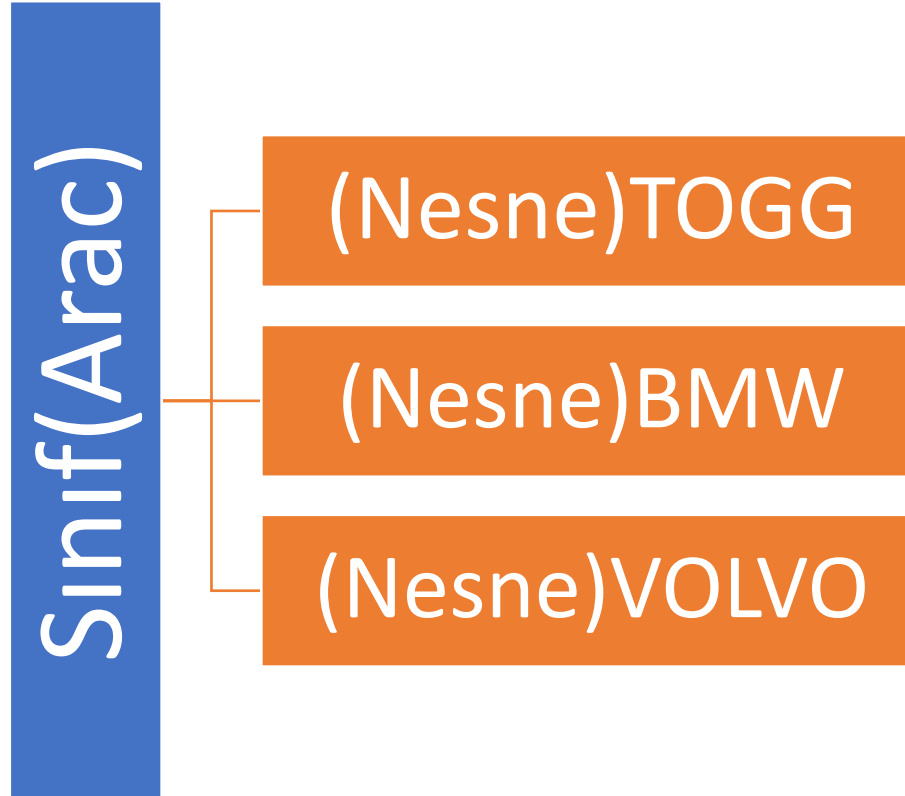
Objects: Real World Examples



Java Sınıflar(Class)

Class()Sınıf

Nesnelerin özelliklerini ve işlevlerini tanımlayan bir şablondur.
Mantıksal bir varlıktır.



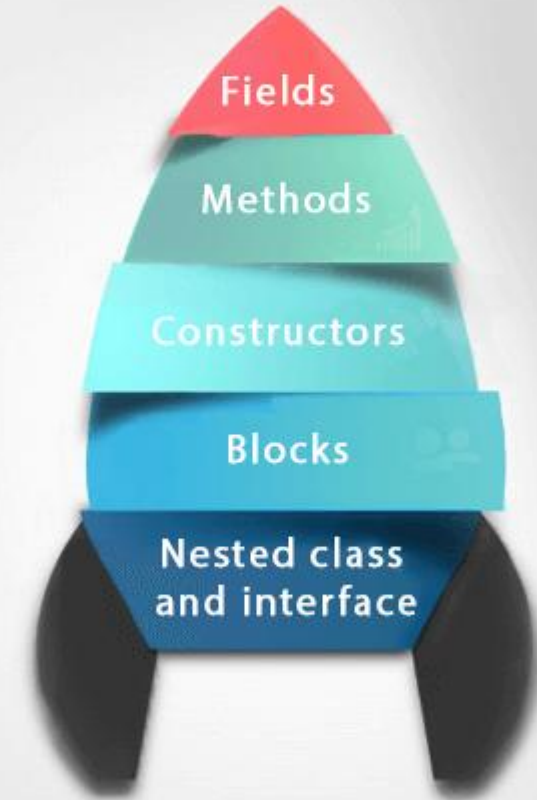
Java Sınıflar(Class)

Sınıf Oluşturma

Nesnelerin özelliklerini ve işlevlerini tanımlayan bir şablondur. Mantıksal bir varlıktır.

```
Erişim_belirteci class <SinifAdi>  
{  
    değişkenler;  
    metotlar;  
    constructor(kurucular);  
    iç içe sınıflar, arayüzler;  
}
```

Class in Java



Java Sınıflar(Class)

Sınıf Oluşturma

```
public class Ogrenci{  
    public int okulNo;  
    public String adSoyad;  
  
}
```

```
//Oluşturulan Ogrenci adlı sınıftan yeni bir ogrenci nesnesi oluşturma  
Ogrenci yeniOgrenci = new Ogrenci();
```

```
//Oluşturulan yeni öğrencinin özelliklerine erişme  
yeniOgrenci.okulNo = 1234;  
yeniOgrenci.adSoyad = "Ali Çalışkan";
```

Java Sınıflar(Class)

Kurucu(Constructor)

Constructor metotlar bir sınıftan bir nesne oluşturulduğunda ilk çalışan metotlardır.

Bu metotlar aracılığı ile sınıflara ilk çalıştıklarında vermek istediğimiz değerleri verebilir, varsayılan değerlere farklı değerler atayabilir veya bir metot çalıştırabiliriz.

Her sınıfın mutlaka varsayılan bir kurucusu(constructor) vardır. Birden fazla kurucu olabilir.

```
public class Ogrenci{  
    int okulNo;  
    String adSoyad;  
}
```

Java Sınıflar(Class)

Kurucu(Constructor)

Kurucu Metot Oluşturma

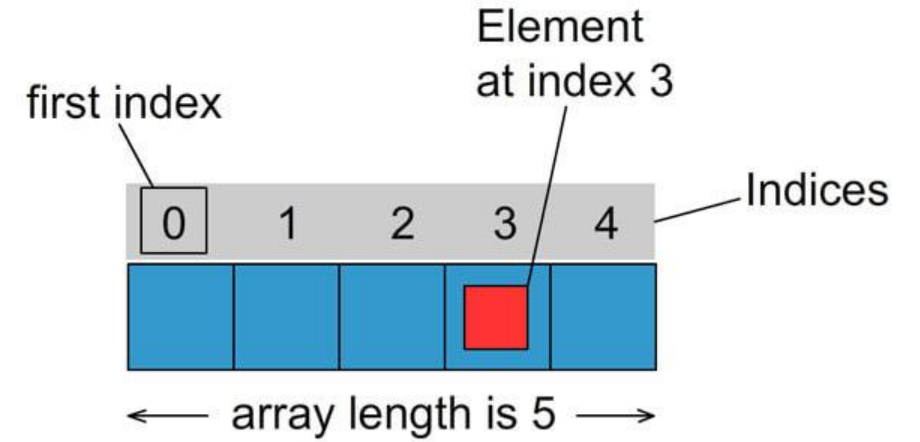
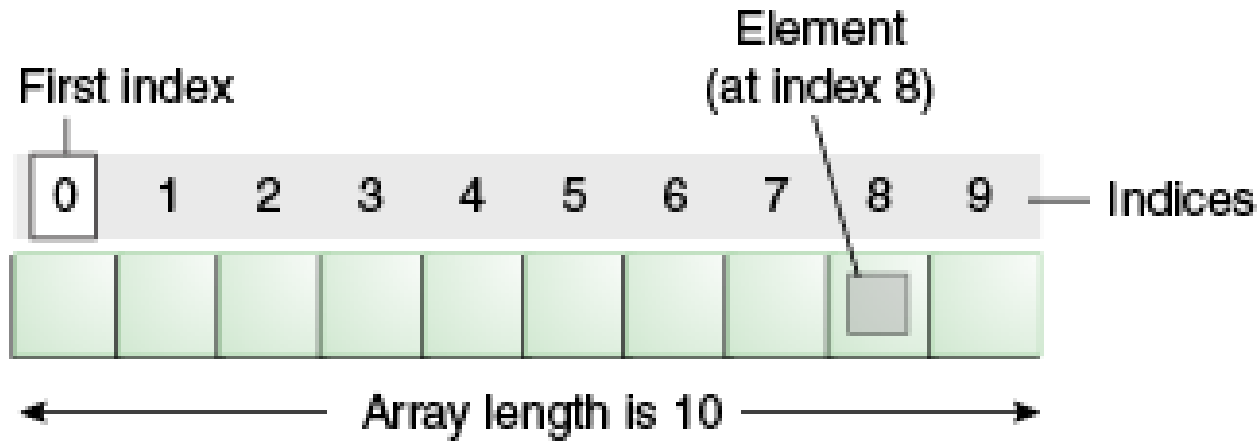
- Kurucu metot sınıf ile aynı adı taşımalıdır.
- Kurucu metotların herhangi bir geri dönüş tipi yoktur.
- Sınıf içerisinde herhangi bir kurucu tanımlanmaz ise varsayılan kurucu çalışır.

```
public class Ogrenci{  
    public int okulNo;  
    public String adSoyad;  
  
    public Ogrenci(){ } //Varsayılan Kurucu  
  
    public Ogrenci(int okulNo, String adSoyad){ //2. Kurucu  
        this.okulNo=okulNo;  
        this.adSoyad=adSoyad;  
    }  
}
```


Java Koleksiyonlar

Diziler(Arrays)

Dizi, benzer türdeki öğelerin tutulduğu bir koleksiyon olarak tanımlanır. Aynı veri tipinden çok sayıda değişkeni tutan yapılara verilen isimdir.



Diziler **0** indis numarası ile başlar.

Örneğin 10 elamanlı bir dizinin son elamanı 9. indise yazılır.

Erişilirken de bu indis numaraları kullanılır.

Java Koleksiyonlar

Diziler(Arrays)

Dizi Tanımlama

```
veriTipi [] diziAdı = new veriTipi [elemanSayısı];
```

```
veriTipi [] diziAdı = {elaman1, elaman2, elaman3} //Başlangıçta değer ataması  
yapılarak tanımlama. Bu durumda dizi boyutu 3 olur.
```

```
veriTipi diziAdı[elemanSayısı]; //Eleman sayısı sonradan da tanımlanabilir.
```

Java Koleksiyonlar

Diziler(Arrays)

Örnek

```
int[] intArray = new int[]{1,2,3,4,5}; // 5 elemanlı önceden değerleri atanmış dizi
```

```
int[] intArray = new int[5]; // 5 elemanlı tanımlanmış değer ataması sonradan yapılmış dizi
```

```
intArray[0]=1; //Dizinin ilk elemanın indis numarası 0 olur  
intArray[1]=2;  
intArray[2]=3;  
intArray[3]=4;  
intArray[4]=5;
```

Dizinin içindeki eleman sayısı(toplam boyut) length yapısı ile öğrenilir.

```
intArray.length // ifadesi 5 değerini döndürür.
```

Java Koleksiyonlar

Diziler(Arrays)

- **Dizinin içindeki elamana erişme:**

Kullanım Şekli: `diziAdi[indisNumarası]=atancak değer;`

Örnek: `intArray[3]=6;` //3 nolu indis değeri 6 olarak değiştirilir

- Dizinin içindeki elaman sayısı(toplam boyut) **length** yapısı ile öğrenilir.

Kullanım Şekli: `diziAdi.length;`

Örnek: `intArray.length` // ifadesi 5 değerini döndürür.

- **Dizileri Sıralama:**

Kullanım Şekli: `Arrays.Sort(diziAdi);`

Örnek: `Arrays.Sort(intArray)` //Dizi küçükten büyüğe sıralanır

Java Koleksiyonlar

Diziler(Arrays)

Dizilerin Eksi Yönleri

- Dizilerin boyutları değiştirilemez.

Örneğin 5 elamanlı bir dizi tanımlandığında fazladan bir elaman eklemeye çalışıldığında out of range(değer sınır dışında) hatası verir.

- Dizilerin bu vb. eksiklerini gidermek için daha gelişmiş koleksiyon sınıfları oluşturulmuştur.

Java Koleksiyonlar

Koleksiyonlar(Collections)

Koleksiyonlar da diziler gibi birden fazla nesne barındırabilen yapılardır.

Farklı amaçlar için kullanılan koleksiyon yapıları vardır. Örnek yapılar: **LIST, MAP, SET**

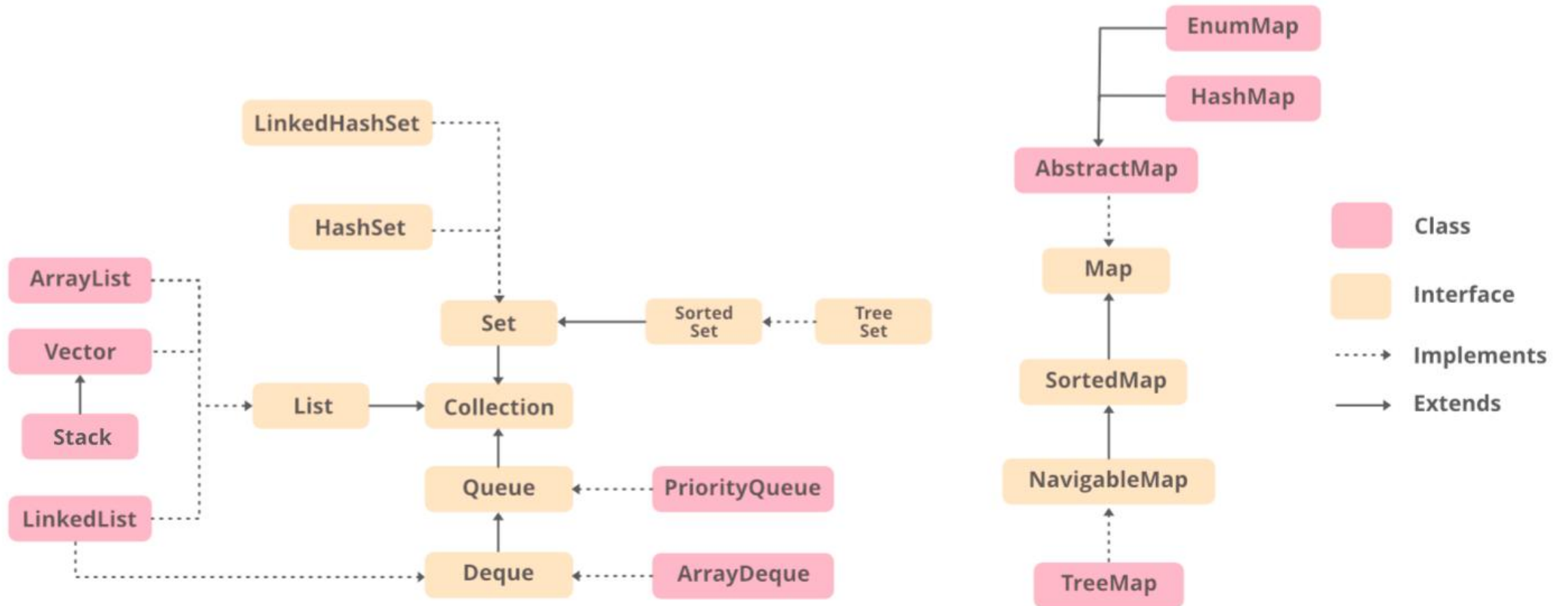
List Nesnesi: Kendisine verilen elemanları sıralı şekilde tutar. Tekrarlanan elemanları barındırabilir.

Set Nesnesi: Kendisine verilen elemanların her birinde sadece bir tanesini tutar. Kopya ya da tekrarlanan elemanları barındırmaz.

Map Nesnesi: Her biri birbirinden farklı anahtarlar ile eşleştirilen nesnelerden oluşur.

Java Koleksiyonlar

Koleksiyonlar(Collections)



Java Koleksiyonlar

Koleksiyonlar(Collections)

ArrayList Yapısı:

- Dinamiktir.
- ArrayList koleksiyon tabanlı bir sınıftır.
- Dizilerin eksiklerini ve kısıtlı olan özelliklerini gidermek için oluşturulmuştur.

Kullanım şekli:

```
ArrayList<veriTipi> koleksiyonAdi = new ArrayList<veriTipi>();
```

Sık Kullanılan Metod ve Yapılar

- **add(atanacakDeğer)** – Koleksiyonun sonuna Eleman eklemek için kullanılır
- **get(indis)** – Koleksiyonun ,içerisindeki istenilen sıradaki elamana erişmek için kullanılır
- **remove(indis)** – koleksiyon içerisindeki belirtilen sıradaki elemanı silmek için kullanılır.
- **set(indis, yeniDeğer)** – koleksiyon içerisindeki istenilen sıradaki elemanın değerini değiştirmek için kullanılır.
- **size()** – koleksiyon içerisindeki eleman sayısını almak için kullanılır.
- **clear()** – koleksiyon içerisindeki elemanların tümünü silmek için kullanılır.
- **contains(aranacakDeğer)** – koleksiyon içerisinde eleman aramak için kullanılır.

Java Koleksiyonlar

Koleksiyonlar(Collections)

Örnek:

```
ArrayList<String> arabalar = new ArrayList<String>();
```

- Listeye eleman ekleme add(atanacakDeğer) metodu ile

```
arabalar.add("Volvo"); //indis: 0
```

```
arabalar.add("BMW"); //indis: 1
```

```
arabalar.add("Ford"); //indis: 2
```

```
arabalar.add("Mazda"); //indis: 3
```

- Listedeki herhangi bir elemana erişmek get(indis) metodu ile

```
arabalar.get(2); // Ford değerini getirir
```

Java Koleksiyonlar

Koleksiyonlar(Collections)

Örnek:

- Listeden elaman silme

arabalar.remove(3); // Mazda değeri siliniz

- Listedeki bir değeri değiştirme

arabalar.set(1,"Mercedes"); // BMW değeri Mercedes ile değişir

- Listenin boyutunu öğrenme

arabalar.size(); // 4 değerini getirir

- Listenin tümünü silme

arabalar.clear();

```
arabalar.add("Volvo"); //indis: 0
arabalar.add("BMW");   //indis: 1
arabalar.add("Ford");  //indis: 2
arabalar.add("Mazda"); //indis: 3
```

Java Koleksiyonlar

Koleksiyonlar(Collections)

List

- List

Kullanım şekli:

```
List<veriTipi> koleksiyonAdi = new List<veriTipi>();
```

Sık Kullanılan Metod ve Yapılar

- **add(atanacakDeğer)** – Koleksiyonun sonuna Eleman eklemek için kullanılır
- **get(indis)** – Koleksiyonun ,içerisindeki istenilen sıradaki elamana erişmek için kullanılır
- **remove(indis)** – koleksiyon içerisindeki belirtilen sıradaki elemanı silmek için kullanılır.
- **set(indis, yeniDeğer)** – koleksiyon içerisindeki istenilen sıradaki elemanın değerini değiştirmek için kullanılır.
- **size()** – koleksiyon içerisindeki eleman sayısını almak için kullanılır.
- **clear()** – koleksiyon içerisindeki elemanların tümünü silmek için kullanılır.
- **contains(aranacakDeğer)** – koleksiyon içerisinde eleman aramak için kullanılır.

View Binding

View Binding nedir

Tasarım ekranındaki nesneleri kod ekranında kullanabilmek için normal durumlarda `findViewById()` metodu kullanılır. Bu durumu kolaylaştırmak için view binding yapısı kullanılarak nesnelere direk isimleri ile ulaşılabilir.

Kullanım şekli:

1. Adım ViewBinding Özelliğini Etkinleştirme:

Android Studio 4.0 ve üzeri versiyonlarda: Gradle içerisine aşağıdaki kod eklenir.

```
buildFeatures {  
    viewBinding = true  
}
```

ÖNEMLİ: Bu ifade yazıldıktan sonra projenin Sync(senkronize) edilmesi yapılmalıdır.

My ApplicationVersion control

Pixel 7 API 33 2app

Android

app

Gradle Scripts

build.gradle.kts (Project: My Application)

build.gradle.kts (Module :app)

proguard-rules.pro (ProGuard rules for

gradle.properties (Project Properties)

gradle-wrapper.properties (Gradle Vers

local.properties (SDK Location)

settings.gradle.kts (Project Settings)

build.gradle.kts (:app)

13applicationId = "com.profile.myapplication"

14minSdk = 28

15targetSdk = 34

16versionCode = 1

17versionName = "1.0"

18

19testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"

20}

21

22buildTypes { this: NamedDomainObjectContainer<ApplicationBuildType>

23release { this: ApplicationBuildType

24isMinifyEnabled = false

25proguardFiles(

26getDefaultProguardFile(name: "proguard-android-optimize.txt"),

27"proguard-rules.pro"

28)

29}

30}

31compileOptions { this: CompileOptions

32sourceCompatibility = JavaVersion.VERSION_1_8

33targetCompatibility = JavaVersion.VERSION_1_8

34}

35

36buildFeatures { this: ApplicationBuildFeatures

37viewBinding = true

38}

39}

40

41dependencies { this: DependencyHandlerScope

42

43implementation("androidx.appcompat:appcompat:1.6.1")

44implementation("com.google.android.material:material:1.11.0")

45implementation("androidx.constraintlayout:constraintlayout:2.1.4")

46testImplementation("junit:junit:4.13.2")

47androidTestImplementation("androidx.test.ext:junit:1.1.5")

48androidTestImplementation("androidx.test.espresso:espresso-core:3.5.1")

49}

MyApplication16 > app > build.gradle.kts > android17:28 LF UTF-8 4 spaces

Windows Taskbar

Search Ara

15:09 23.02.2024

View Binding

Kullanım şekli:

2. Adım ViewBinding Özelliğini Activity içesinden çağırma ve nesnelere erişme:

```
public class MainActivity extends AppCompatActivity {

    ActivityMainBinding binding; //ActivityMainBinding türünde bir değişken tanımlıyoruz

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        //setContentView(R.layout.activity_main); //eski yöntemde tasarım dosyasına bu şekilde ulaşılır
        binding = ActivityMainBinding.inflate(getLayoutInflater()); //yukarıda oluşturulan değişkene .inflate metodu ile atama yapıyoruz
        setContentView(binding.getRoot()); //setContentView metoduna tanımladığımız ve gerekli atamayı yaptığımız tasarımdaki kök
        elemanı olarak tüm tasarımı atıyoruz
        //Artık nesnemize belirtilen yapı üzerinden ulaşabiliriz
        binding.textViewBilgi.setText("Bilgi");
    }
}
```

ÖNEMLİ: Birden fazla activity kullanıyorsanız kullandığınız activitye verdiğiniz isme göre **ActivityMainBinding** yerine, o sayfaya verdiğiniz ada göre bir tanımlama oluşur. Örneğin: DetayActivity.java ve activity_detay.xml dosyalarından oluşan bir activity için tanımlama **ActivityDetayBinding** şeklinde bir çağırma yapılmalıdır.